

A Novel Hybrid Model for Docker Container Workload Prediction

Liangkang Zhang, Yulai Xie, *Member, IEEE*, Minpeng Jin, Pan Zhou, Gongming Xu, Yafeng Wu, Dan Feng, *Senior Member, IEEE*, Darrell Long, *Fellow, IEEE*

Abstract—The emergence of containers dramatically simplifies and facilitates the development and deployment of applications. More and more enterprises deploy their applications on the container cloud platform. For cloud service providers, an effective container workload prediction method is a must to achieve efficient utilization of cloud resources. However, the existing methods are either rarely based on container load characteristics or cannot make accurate real-time predictions. In this paper, we propose a Docker container workload proactive prediction method using a hybrid model combining triple exponential smoothing and long short-term memory (LSTM), which not only can capture both short-term and long-term dependencies in container resource time series but also smooth the container resource utilization data. In order to improve the prediction accuracy of the hybrid model, those two single models are combined using the mean absolute percentage error (MAPE) method. Besides, we design a real-time Docker workload prediction system for the hybrid model. Our experiments show that the mean absolute percentage error of the hybrid model is decreased by an average of 3.24%, 12.18%, 13.42%, 43.45%, and 50.69% compared with the LSTM, the triple exponential smoothing, ES-ARIMA, Bayesian Ridge Regression and BiLSTM with an acceptable time and computational cost overhead.

Index Terms—Docker, Workload prediction, LSTM, Triple exponential smoothing

I. INTRODUCTION

THE lightweight Docker containers greatly simplify and facilitate the development and deployment of applications [1], and the elastic scalability and pay-as-you-use features of the cloud platform can reduce the economic burden of users. The need for virtualization of the popular microservices makes containers increasingly popular [2]. IoT and edge/fog computing systems also need containers to architect [3]. Therefore, more and more companies deploy their applications on the container cloud platform.

To ensure the normal running of Docker containers, we must allocate sufficient resources properly. However, the character-

istics of the Docker container, such as poor resource isolation, short running period, and elastic scaling, pose a greater challenge and a higher requirement for accurate resource prediction and provisioning. There are some studies on container performance [4]–[8] and container resource management [9]–[11]. A good load prediction method can effectively improve resource management capabilities and ensure container performance. For cloud service providers, an accurate load prediction method is a must to improve resource utilization, achieve efficient elastic scaling, and ensure pay-as-you-go service.

The resource provisioning methods can be classified into two categories: reactive and proactive. The former is to allocate resources after the resource demands arrive, while the latter allocates resources in advance [12]. The proactive method allocates resources in advance by predicting the coming resources. Therefore, the proactive method can provide resources much faster than the reactive method when massive resource requirements come, which also puts forward higher requirements and more significant challenges for the proactive method. In the proactive container load prediction scenario, we must consider the following factors. First, the resource provisioning should be as close to the actual resource demands as possible. Second, applications in container environments have fluctuating resource demands resulting in complex behavior in resource utilization [13]. Third, the workload patterns generated by different types of applications are different, which poses a considerable challenge for accurate proactive workload prediction.

To improve the accuracy of container workload prediction, it is vital to analyze the characteristics of the container load. We observe the container resource utilization for different container workloads as shown in Figure 1. The data patterns of the container resource utilization generated by different types of containers are different. Some have short-term and long-term repetitive patterns, while some do not. According to the observation results, the prediction method should consider both the repetitive load characteristics and the variation trend of non-repetitive loads and be able to adjust dynamically according to the load changes.

Whether it is a virtual machine or a container, there will be repetitive patterns in its workload [16], [17]. In comparison to virtual machines, the granularity of container resource allocation is finer, and the startup and destruction costs are smaller [18], resulting in container load fluctuating more randomly and the amount of training data collected being smaller. In this case, many methods [19]–[22] used for VM load prediction may not apply to container load prediction.

Liangkang Zhang, Yulai Xie, and Yafeng Wu are with the Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China (email: lkzhang98@gmail.com, {ylxie, yafengwu}@hust.edu.cn)

Minpeng Jin, Gongming Xu, and Dan Feng are with the School of Computer, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, P.R. China (email: {jinminpeng0510, xugongming38}@gmail.com, dfeng@hust.edu.cn)

Pan Zhou is with Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, P.R. China (email: panzhou@hust.edu.cn)

Darrell Long is with Jack Baskin School of Engineering, University of California, Santa Cruz, CA 95064 USA (email: darrell@ucsc.edu)

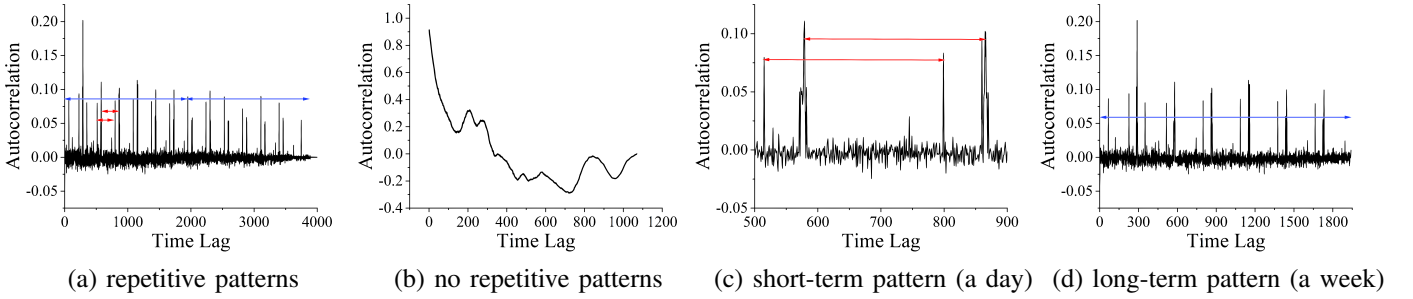


Fig. 1: Autocorrelation graphs for randomly selected container data from Alibaba dataset [14] as shown in Figure (a) and Figure (b). Autocorrelation, called serial correlation, is the correlation of a signal with its delayed copy [15]. Every time lag is 5 minutes in the datasets and we enlarge the short-term and long-term repetitive patterns of Figure (a) as shown in Figure (c) and Figure (d). The short-term pattern in Figure (c) has a repetitive pattern in about 288 time lags (a day) as indicated by the red two-way arrow. And we can also observe a long-term repetitive pattern in about 1950 time lags (about a week) as indicated by the blue two-way arrow in Figure (d). In Figure (b), there are no short-term and long-term repetitive patterns.

So far, there are only a few related researches on Docker workload prediction from the perspective of container workload data patterns. Xie *et al.* [23] proposed a hybrid model combining the ARIMA model and triple exponential smoothing model. It can capture both linear and nonlinear relationships in the container resource load sequence. But it cannot dig other relationships. Chen *et al.* [24] presented a container workload prediction approach called PSL which uses LSTM to conduct prediction. But the prediction process is too complicated to predict in real-time. Moreover, offline LSTM fails to adjust in real-time based on load data and eliminate fluctuations in container load.

To fully exploit the characteristics of container workload and achieve accurate real-time container workload prediction, we propose a novel hybrid Docker container workload prediction method combining triple exponential smoothing and LSTM. LSTM can exploit the short-term and long-term patterns in workload data by using the gate mechanism to reset its state [25]. Triple exponential smoothing [26] can capture the trends in data and smooth the container load fluctuations by allocating different weights to the historical data. We combine the two models using the mean absolute percentage error (MAPE) [27] method. Besides, we design and implement a real-time container workload prediction system that can collect and calculate resource utilization from containers.

The contributions of this paper are as follows:

- We design a Docker container workload proactive prediction method using a hybrid approach of triple exponential smoothing and LSTM. We design the hybrid model as an online update model by dynamically computing the historical error of each single model and setting their weights.
- We design a real-time Docker container workload prediction system for the hybrid model which can effectively collect the Docker performance metrics and perform an accurate prediction.
- We evaluate the hybrid model using both a simulated cloud environment and real public datasets. The results of the experiment demonstrate the good prediction performance of the hybrid model with the acceptable time and computational cost overhead.

II. BACKGROUND AND MOTIVATION

We elaborate on triple exponential smoothing and LSTM and explain the motivation behind them.

A. Triple exponential smoothing

The triple exponential smoothing [26] is a common method for predicting time series. It uses a weight that falls off exponentially as the historical data gets older [28].

The prediction process of the triple exponential smoothing is divided into three steps. Firstly, according to the current real value and the smoothed values for the foregone time, calculate the smoothed values for the current time. The formulas for calculating the smoothing values are as follows:

$$S_t = \alpha x_t + (1 - \alpha)S_{t-1} \quad (1)$$

$$R_t = \alpha S_t + (1 - \alpha)R_{t-1} \quad (2)$$

$$G_t = \alpha R_t + (1 - \alpha)G_{t-1} \quad (3)$$

α refers to the smoothing factor, x_t represents the real value of the present time, and S_t , R_t , and G_t are the smoothed value, quadratic exponential smoothed value, and triple exponential smoothed value. Secondly, the coefficients are calculated according to the smoothed values. The formulas are as follows:

$$x_t = 3S_t - 3R_t + G_t \quad (4)$$

$$y_t = \left(\frac{\alpha}{2(1 - \alpha)^2}\right)[(6 - 5\alpha)S_t - 2(5 - 4\alpha)R_t + (4 - 3\alpha)G_t] \quad (5)$$

$$z_t = \left(\frac{\alpha^2}{2(1 - \alpha)^2}\right)[S_t - 2R_t + G_t] \quad (6)$$

Finally, the prediction value can be obtained as shown in Formula 7. m is the m -th time to be predicted from the present time.

$$F_{t+m} = x_t + y_t m + z_t m^2 \quad (7)$$

It can be seen from the above expressions that the triple exponential smoothing uses exponentially weighted processing of historical data, which allows it to smooth load data well and capture the load data trends. When the triple exponential smoothing is used for the time series of container performance

metrics, it only needs to store the smoothed values for the foregone time and predict container performance metrics at the next time with the current actual value of the container performance metrics. So it has a minimal time overhead for time series prediction, and it is easy to be implemented as an online update prediction model.

B. Long short-term memory

The LSTM neural network model [29] can predict the complex long-term dependence and advanced seasonal dependence in time series. The network structure of LSTM has special memory cell units. LSTM unit comprises a cell, input gate, output gate, and forget gate, as shown in Figure 2. x_t represents the time series data of the input layer at the time t , h_{t-1} and C_{t-1} represent the hidden state and cell state of the LSTM unit at the foregone time, respectively. There are two outputs of the LSTM unit: one is the LSTM cellular state C_t at the current moment, and the other is the hidden state h_t at the current moment.

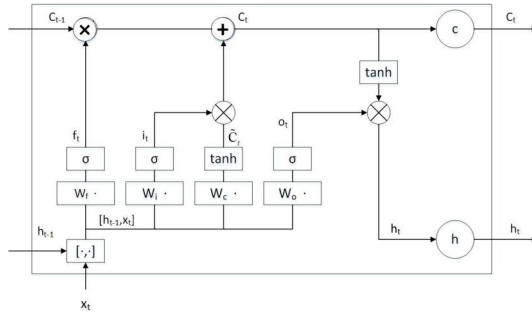


Fig. 2: LSTM unit structure graph. The LSTM comprises memory cell units that enable the exploitation of short-term and long-term temporary dependencies in time series.

The update formulas of the LSTM unit are as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (8)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (9)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (10)$$

$$C_t = C_{t-1} \cdot f_t + i_t \cdot \tilde{C}_t \quad (11)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (12)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (13)$$

i_t represents the input gate. W_i , W_c , b_i , b_c respectively represent the weight coefficients and biases of each part of the input gate. W_o , b_o , W_f , b_f represent the weight coefficient and bias of the output gate and forget gate respectively.

We use LSTM to exploit short-term and long-term temporal dependencies in container resource data. Furthermore, when containers have been running for some time, the running state may continuously change with time. The trained LSTM model can no longer accurately predict the current running state. It is necessary to use the newly collected real-time data to update the model to adapt to the current running state of containers. Only by this can the LSTM model constantly adapt to the

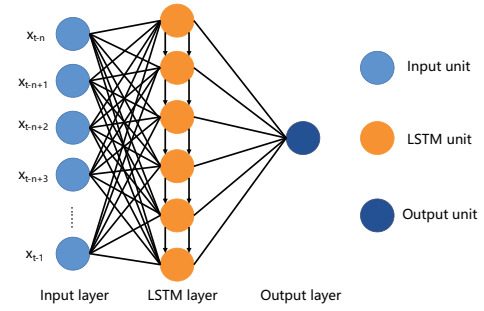


Fig. 3: Schematic diagram of our proposed dynamically updated LSTM network.

dynamic changes of data and ensure a superior prediction accuracy of the model.

III. DESIGN OF THE PREDICTION METHOD

In this section, we first describe the reason for combining LSTM and triple exponential smoothing. Then we detail the design of the hybrid model.

A. The reason for combining LSTM and triple exponential smoothing

Firstly, from the perspective of container load characteristics, LSTM can store short-term and long-term data characteristics and predict complex short-term and long-term temporal dependencies in container load data. However, LSTM cannot handle container load fluctuations resulting in the predictions being unstable. Triple exponential smoothing can smooth fluctuations well in the container environment by exponentially weighting the historical data. By combining these two models, the hybrid model can deal with more load patterns and achieve higher prediction accuracy.

Secondly, from the perspective of the prediction period, LSTM is a neural network model that requires a large amount of load data for training. When the amount of data is relatively small, triple exponential smoothing can compensate for the low prediction accuracy caused by LSTM due to less training data. As the data volume increases, the prediction accuracy of LSTM will become higher. In this way, the hybrid model can have good prediction performance in the case of both small and large data volumes.

B. Long short-term memory network

The network of LSTM we use is shown in Figure 3. The input layer of the network is $\{x_{t-n}, x_{t-n+1}, \dots, x_{t-1}\}$ at time t whose size is n . To reduce the model's overhead, we use only six LSTM units to build the LSTM layer. The output layer is a fully connected layer with a unit to output the prediction result.

When the data pattern of the resource utilization changes, using the trained LSTM to perform prediction will result in a decrease in prediction accuracy. Therefore, LSTM needs to be updated online in real-time to improve prediction accuracy. We use Stochastic Gradient Descent (SGD) [30] to update LSTM. The latest data point of resource utilization is used to update the LSTM weights W_t to minimize the loss function.

The input features of LSTM is $\{x_{t-n}, x_{t-n+1}, \dots, x_{t-1}\}$ at time t , the prediction value at the next moment is $\tilde{x}_t = g(x_{t-n}, x_{t-n+1}, \dots, x_{t-1}; W_{t-1})$. The LSTM weights are updated using the SGD and the formulas are as follows:

$$W_t = W_{t-1} - \eta \nabla Q_t(W_{t-1}) \quad (14)$$

$$Q_t(W_{t-1}) = \frac{1}{2}(x_t - \tilde{x}_t)^2 \quad (15)$$

$$\nabla Q_t(W_{t-1}) = -(x_t - \tilde{x}_t) \nabla g(x_{t-n}, x_{t-n+1}, \dots, x_{t-1}; W_{t-1}) \quad (16)$$

Where η is the learning rate of the model. $Q_t(W_{t-1})$ is the loss function which is the mean square error function. $\nabla Q_t(W_{t-1})$ is the gradient of the loss function. The stochastic gradient descent method can achieve the online update process of LSTM.

C. Prediction using the hybrid model

1) *The MAPE combination method:* The MAPE method [27] is defined as follows: let the data of a time series data be $\{x_1, x_2, x_3, \dots, x_n\}$, m single prediction models are used to predict the sequence, and the prediction value of the k -th prediction model at time t is $\hat{x}_{kt}$. Then the prediction error of the k -th prediction model is $e_{kt} = (x_t - \hat{x}_{kt})$. The prediction accuracy A_{kt} of the k -th prediction model at time t is:

$$A_{kt} = \begin{cases} 1 - \left| \frac{e_{kt}}{x_t} \right|, & \left| \frac{e_{kt}}{x_t} \right| < 1 \\ 0, & \left| \frac{e_{kt}}{x_t} \right| \geq 1 \end{cases} \quad (17)$$

The mean prediction accuracy d_k of the k -th model is:

$$d_k = \frac{1}{n} \sum_{t=1}^n A_{kt} \quad (18)$$

The combination method of two models determines whether the hybrid model can fully exploit the strengths of both models. We use the MAPE method to combine those two models and set the initial weight to 0.5. Then we dynamically calculate and update subsequent weight as shown in Algorithm 1.

In Algorithm 1, sum_tes and sum_lstm are the sum of the prediction accuracy of triple exponential smoothing and LSTM, respectively. acc_lstm_t and acc_tes_t are the prediction accuracy at time t calculated by the Formula 17. Since the number of time points predicted by the two models is the same, the ratio of the mean prediction accuracy of the two models to the sum of the prediction accuracy is the same. So we use the sum of prediction accuracy instead of the mean prediction accuracy to calculate the weights.

2) *Prediction process of the hybrid model:* Real-time workload prediction first collects a small part of data from the system in advance as pre-training data, then continuously collects load from the system at intervals of seconds and feeds back the predicted results to the system. The real-time hybrid model prediction process contains the four steps below and the detail is shown in Figure 4.

1) Triple exponential smoothing and LSTM are implemented as online learning prediction models. We use

Algorithm 1 get_weight

Input: $Predict_lstm_t, Predict_tes_t, x_t$ // $Predict_lstm_t$ refers to the prediction result of LSTM at current time point, $Predict_tes_t$ refers to the prediction result of triple exponential smoothing at current time point. x_t refers to the current real value.

Output: $weight_tes, weight_lstm$ // $weight_tes$ refers to the weight factor of the triple exponential smoothing next time and $weight_lstm$ refers to the weight factor of the LSTM next time.

- 1: $acc_lstm_t \leftarrow A_{kt}(Predict_lstm_t, x_t)$
- 2: $acc_tes_t \leftarrow A_{kt}(Predict_tes_t, x_t)$
- 3: $sum_lstm \leftarrow sum_lstm + acc_lstm_t$
- 4: $sum_tes \leftarrow sum_tes + acc_tes_t$
- 5: $weight_tes \leftarrow sum_tes / (sum_tes + sum_lstm)$
- 6: $weight_lstm \leftarrow sum_lstm / (sum_tes + sum_lstm)$
- 7: **return** $weight_tes, weight_lstm$

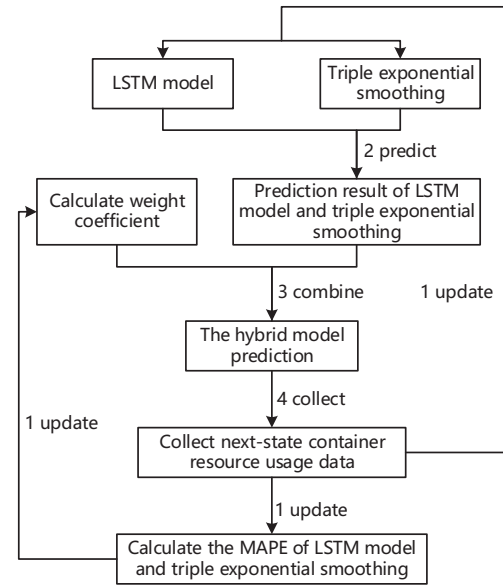


Fig. 4: Hybrid model prediction process.

the MAPE method to combine these two models into a hybrid model. We use the real load value to update the LSTM model and triple exponential smoothing. Then we calculate the mean absolute percentage error of the single model and set the weight factor of the single model.

- 2) Prediction result of the LSTM model and triple exponential smoothing for the next point is generated.
- 3) Use the weight factor and prediction result of two models to generate the hybrid model prediction value.
- 4) Obtain the real value of the container load utilization the next time and return step 1 to update the model and weight factor.

The hybrid model combination process is shown in Algorithm 2. $Predict_tes_{t+1}$ and $Predict_lstm_{t+1}$ refer to the prediction result of the triple smoothing and LSTM the next time. $weight_tes$ and $weight_lstm$ refer to the weight factor of the LSTM and triple exponential smoothing, respectively.

Algorithm 2 Hybrid model prediction process

Input: x_t // x_t refers to current real value.

Output: *Predict_result* // The prediction result of the hybrid model.

- 1: $(weight_tes, weight_lstm) \leftarrow get_weight(Predict_lstm_t, Predict_tes_t, x_t)$
- 2: $Predict_lstm_{t+1} \leftarrow LSTM.predict()$
- 3: $Predict_tes_{t+1} \leftarrow TripleExpSmoothing.predict()$
- 4: $Predict_result \leftarrow Predict_lstm_{t+1} \times weight_lstm + Predict_tes_{t+1} \times weight_tes$
- 5: **return** *Predict_result*

IV. PREDICTION SYSTEM FOR HYBRID MODEL

We introduce the design and implementation of the real-time Docker container workload prediction system for the hybrid model.

A. Architecture

The Docker container workload prediction system is composed of four modules as shown in Figure 5.

- 1) *Container load collection* module uses the Docker API to obtain information about all running containers on the machine and their resource utilization and transmits them to *container load data storage* module.
- 2) *Container load data storage* module first performs data cleaning on each resource utilization data to delete duplicate values and fill in loss values of the data and then stores the load data into the database. The data is also sent to *container load prediction* module for further analysis.
- 3) *Container load prediction* module encapsulates the hybrid model of triple exponential smoothing and LSTM. Before the module makes predictions, the hybrid model is pre-trained using the data obtained from the *container load data storage* module. After pre-training, it performs online prediction and then obtains the real value. The prediction value is sent to *container load scheduling* module. In the end, the hybrid model is updated based on the real value.
- 4) *Container load scheduling* module dynamically schedules container resource utilization through the predicted value sent by the *container load prediction* module.

B. Container load collection

Container load collection module needs to perceive information such as the start and stop of the containers, so as to correctly collect resource utilization data. The module is divided into the following three components.

- 1) *Container information management*: *Container information management* component obtains the container states through the Docker API and obtains container information on the host by analyzing its start event and stop event. Container information includes container ID, process number, and task information. The component maintains a container list *ContainerInfoList* as shown in Table I.

TABLE I: Element definition of the ContainerInfoList

Field name	Type of data	Description
ContainerID	char[64]	Container ID
ProcessNumber	unsigned int	Container process number
ContainerTask	char[256]	Container startup task

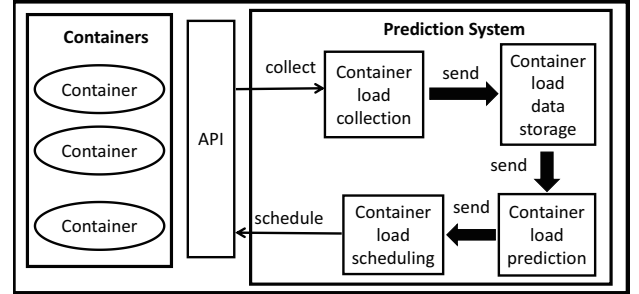


Fig. 5: System design architecture.

2) *Data collection control*: We consider container workload prediction from the perspective of time series prediction and process the collected resource utilization data as time-series data. Therefore, a fixed time interval is required to monitor the containers, that is, the monitoring cycle of the container. We use the fixed monitoring cycle for all containers. The component selects the container to be collected by *monitoring data collection* component according to the next data collection time. We use a priority queue to implement container selection. The data item with the earliest next collection time is placed at the head of the priority queue. The data items in this queue are referred to as the CollectItem structure as shown in Table II.

TABLE II: CollectItem structure

Variable	Type of data	Description
ContainerID	char[64]	Container ID
ProcessNumber	unsigned int	Container process number
Next_time	time.Time	The next data collection time

3) *Monitoring data collection*: We collect the resource utilization data by reading the *cgroups* file and the */proc* virtual file system. We can also use *glances* [31] for collection, which supports a direct collection of container resource utilization and provides an interface for storage to the database. Regarding the interval of capturing load, we recommend setting seconds as the intervals for collection, storage, prediction, and scheduling to achieve accurate real-time load prediction. The experimental results prove that hybrid model can meet the real-time prediction with the interval in seconds. The calculation method of resource utilization is as follows:

- 1) *CPU utilization*: According to the amount of CPU time slice used in the last monitoring cycle of the container (CPU_used) and the total amount of time slice used in the last monitoring cycle of the host (CPU_total), we can get CPU utilization of the container. The total amount of CPU time slice consumed by the host since startup can be obtained in the

host's `/proc/stat` directory and the CPU time slice consumed by the container since startup can be found in the host's `/sys/fs/cgroup/cpuacct/docker/$containerID/cpuacct.usage` file. Besides, when the host machine has a multi-core processor, we need to multiply the number of cores as shown in Formula 19.

$$U_{cpu} = cores \times \frac{CPU_used}{CPU_total} \times 100\% \quad (19)$$

2) *Memory utilization*: The memory utilization can be computed by using `Memory_used` to divide by `Memory_limit` as shown in Formula 20. `Memory_used` is stored in the `/sys/fs/cgroup/memory/docker/$containerID/memory.usage_in_bytes` file, and `Memory_limit` is recorded in the `/sys/fs/cgroup/memory/docker/$containerID/memory.limit.in_bytes` file.

$$U_{memory} = \frac{Memory_used}{Memory_limit} \times 100\% \quad (20)$$

TABLE III: Dirty data types and description

Type	description
Loss data	The data monitored at a certain moment is null
Duplicate data	The same value appears multiple times at the same time in a row

C. Container load storage

1) *Data cleaning*: Many containers that need to be monitored on the host machine, and there may be multiple containers starting and stopping at the same time. The problems of data loss and data duplication are inevitable. Therefore, before storing in the database, the collected resource utilization data need to be cleaned to remove dirty data. The types of dirty data are shown in Table III.

The duplicate data in the resource utilization data will have an impact on the update of *container load prediction* module, resulting in a sharp decrease in the prediction accuracy of the prediction model. If the collected data has the same container ID and the same timestamp, we only keep one of them. Besides, severe data loss will also have a significant impact on the update of the prediction model. This is because the lost data will cause the loss of continuity of the collected resource utilization data, resulting in the update of the prediction model deviating from the normal mode. Especially when the state of the container changes, loss data will cause the prediction model to fail to adapt to the new data distribution in time, resulting in a large number of false alarms. If the data loss is too serious, the data currently collected should be discarded and should be collected again. However, if the data loss is relatively minor, multi-level Lagrangian interpolation is used to interpolate to fill in the loss data S_t . The formula is:

$$S_t = \frac{\sum_{k=1}^{n_1} S_{t-k} + \sum_{k=1}^{n_2} S_{t+k}}{n_1 + n_2} \quad (21)$$

where n_1 and n_2 represent the length of time before and after the present time point respectively.

2) *Database storage*: After the resource utilization data are cleaned, we store them in the database. We store the data instead of directly sending it to *container load prediction* module. As the pre-training part of the prediction model requires sufficient offline data to train the prediction model, we import the offline dataset into the database and perform the prediction. We use InfluxDB [32] for storage. This database is an open-source distributed time series, event, and indicator database that meets the storage requirements of resource utilization data. There are two types of tables in the database, the container information control table, and the container monitoring data table.

We create a container monitoring data table for each monitored container which stores the resource utilization data and uses the container ID as the table name. The table stores the timestamps of the collection time, the container's CPU utilization, and memory utilization, and uses the timestamps as the primary key and sorts by timestamps. The container information control table stores container information data. The information includes the container ID, the time when the container is last updated, and the number of data the container has collected. Through this table, we can quickly query the data collected.

D. Container load prediction

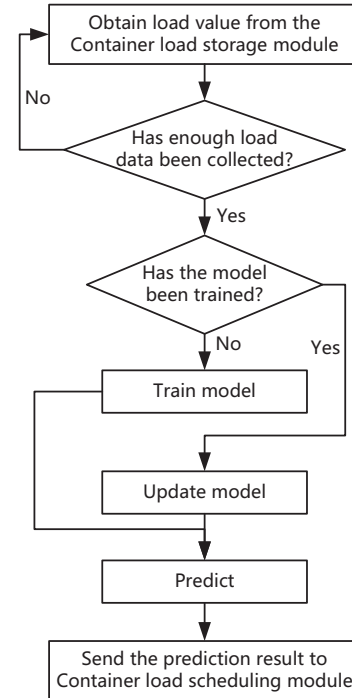


Fig. 6: Container load prediction module prediction process.

Container load prediction module encapsulates the hybrid model based on LSTM and triple exponential smoothing. The resource utilization data sent from *container load data storage* module are processed into a certain format, and then a part of offline data is used to pre-train the hybrid model. After pre-training, we perform online prediction and then obtain the real value. When a new container is started, a

new thread will be responsible for load prediction for the container until the container is closed. The prediction process is shown in Figure 6. The characteristics and resource metrics of different loads are different, so we need different prediction parameters for different resource types. If we want to predict two resources at the same time, we need to open two threads at the same time.

Before the hybrid model is used to predict, it needs to collect resource utilization data for a while for offline pre-training of the hybrid model, which allows the hybrid model to learn container-generated data patterns' characteristics. After pre-training, triple exponential smoothing and LSTM perform prediction, respectively, and the MAPE method is used to obtain the weights of the two models. The predicted value of the hybrid model is sent to *container load scheduling* module. Ultimately, this module implements the online update of the triple exponential smoothing and LSTM model.

Regarding the amount of data to be used for training, choosing too little data may result in a poor prediction effect at the beginning of the model. Choosing too much will result in a long time to produce prediction results. According to our experiments' results and experience, we believe it is more appropriate to use the load data of about one hundred time points as the pre-training data.

The resource utilization data sent by *container load data storage* module can be directly used to update triple exponential smoothing. However, LSTM is a supervised learning model as a neural network model. Moreover, the resource utilization data must be processed into a data format suitable for LSTM. LSTM generally needs to normalize the data to $(0, 1)$. However, our proposed prediction algorithm is an online algorithm, which cannot accurately obtain the maximum and minimum for data preprocessing. We can pre-estimate the maximum and minimum from the max and min values of the training data. Since the load may vary within a range, the pre-estimated maximum and minimum may be inaccurate. So we do not perform data preprocessing. The experimental results show that the prediction accuracy of the hybrid model is acceptable without data preprocessing during real-time prediction.

E. Container load scheduling

The purpose of hybrid model prediction is to improve the utilization rate of resources, so it is necessary to use the prediction result as a resource reference to schedule container resources. *Container load scheduling* module receives the predicted value sent by the *container load prediction* module and dynamically schedules and allocates container resources. Although the prediction accuracy is relatively high, the predicted value cannot be used directly as the container resource usage. This is because there is still an error between the predicted and actual values. Assigning the predicted value directly as the resource limit will affect the performance of the container. We use the range of load resources $(Max(x_t) - Min(x_t))$ as a possible prediction error. The resource limit value can be calculated as follow:

$$Limit = prediction + (Max(x_t) - Min(x_t)) \quad (22)$$

Where *prediction* is the prediction result sent by *container load prediction*, *Limit* is the value allocated to container resource limit, and x_t is the container workload for prediction.

For CPU resource allocation, when the *Limit* of CPU utilization is calculated, we can set the CPU utilization of the container by executing the *docker update --cpu-shares cpu_share containerID*, where *--cpu-shares* is the relative weight of CPU shares. We also can execute the *docker update --cpu-quota cpu_quota --cpu-period cpu_period containerID* to set an absolute limit on CPU utilization, where *--cpu-period* refers to the CPU CFS scheduler period of the container and *--cpu-period* refers to the CPU CFS quota of the container.

The maximum allocation value can be set by the *cgroups* file when a container is created for memory resource allocation. We can execute the *docker update --memory memory_limit containerID* to dynamically limit the memory utilization, where *--memory* is the limit of memory.

V. EXPERIMENT

In this section, we first introduce the experiment environment. Then we present how to determine the parameters of the hybrid model and compare the prediction accuracy of the hybrid model with several workload prediction methods. Finally, we analyze the time overhead, computational cost overhead, and scalability of the hybrid model.

A. Experimental environment

We not only perform experiments in a simulated cloud environment but also use the public real dataset that is collected in the real cloud experiment. For the simulated environment, we collect the resource utilization of three popular containers. For the public real dataset, we use the Alibaba cluster-trace-v2018 [14] to represent container load in the real cloud environment.

TABLE IV: Configuration detail of the simulated environment

Platform	Hardware	Application
Ubuntu20.04.3	Intel i7-7700	Redis 6.2.6
Docker20.10.9	CPU@3.60GHz	Nginx 1.18.0
	4Cores, 8GB RAM	Mysql 5.7.35

The configuration of the simulated environment is shown in Table IV. The simulated container cloud platform has 4 CPU cores and 8 GB RAM, running Ubuntu 20.04.3 and Docker 20.10.9. We select three popular application containers to collect container workload data. The first application container is Redis [33], an open source key-value pair storage database that supports network, memory-based, and distribution. In our experiment, Redis acts as a cache service.

The second is Nginx [34], which is a lightweight web server and reverse proxy server with high performance. In our experiment, Nginx acts as a web server.

The third is Mysql [35], which is a popular cross-platform and efficient relational database. In our experiment, Mysql acts as a database service.

We use an open-source website project [36] to build a website using these three application containers. The access to

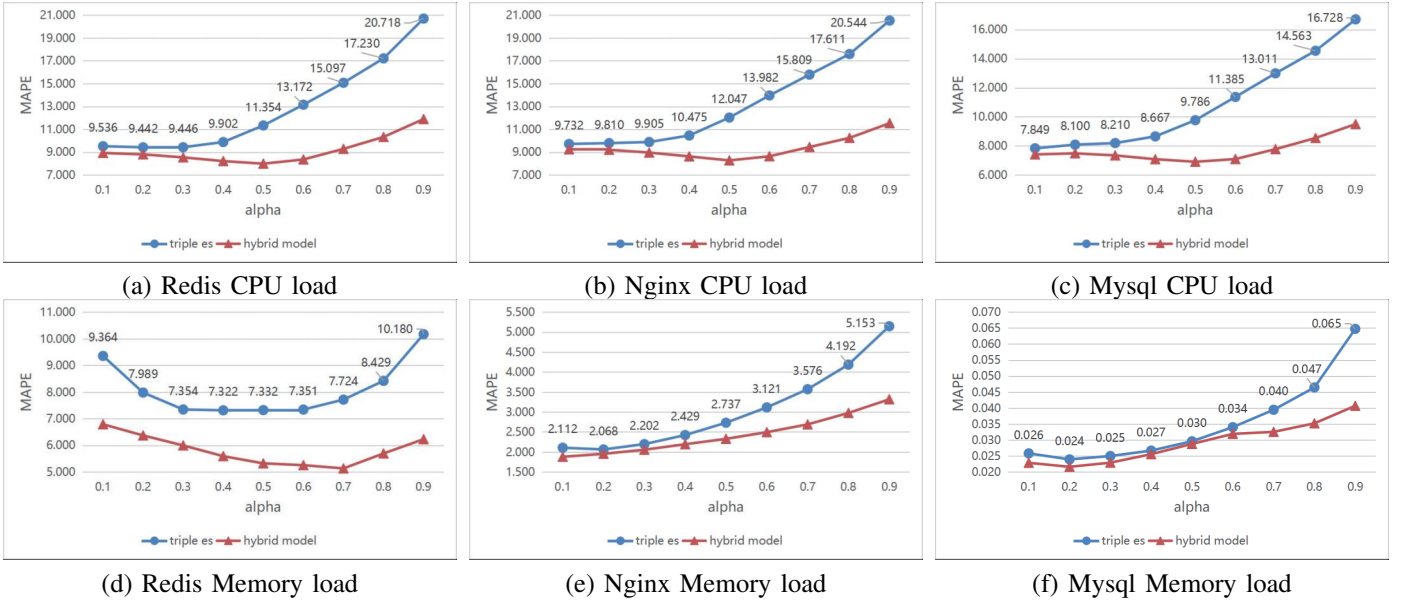


Fig. 7: The MAPE values of the hybrid model and triple exponential smoothing under diverse smoothing factors.

the website will trigger the containers to collect data and we can also capture the interaction between the containers. This is close to the real cloud platform. We use the HTTP stress tool `http_load` [37] to simulate multiple user access and use `glances` [31] to collect the resource utilization of containers. We execute the command `glances -t 1 --export csv --export-csv-file docker_workload.csv`, which collects container workload information every second and save it to `docker_workload.csv` in csv format. CPU and memory are the two most common resources, so here we use the CPU and memory utilization of three applications for the experiment.

In addition, we also use the Alibaba cluster-trace-v2018 [14] to further evaluate the hybrid model. Alibaba cluster-trace-v2018 dataset records the load of about 4000 machines in a period of 8 days. The cluster-trace-v2018 consists of machine usage, machine meta, container meta, container usage, batch task, and batch instance [38]. For container usage data, every container corresponds to a machine, so the machine usage is consistent with container usage. Because we study the load of the container, we use all of the container workload usage data. As a lot of data is not equally spaced in the Alibaba cluster-trace-v2018, we resample the trace data into 10-minute intervals and use interpolation to complete the missing values.

B. Parameters of the hybrid model

The parameter of triple exponential smoothing: for triple exponential smoothing, the value of the smoothing factor α is crucial to the triple exponential smoothing prediction accuracy. The smoothing factor decides the proportion of new data and original prediction data. The smaller the smoothing factor, the lesser the proportion of new data. The experiment takes the smoothing factor α as 0.1 ~ 0.9 respectively and determines the optimal value of the smoothing factor according to the mean absolute percentage error (MAPE). The smaller the MAPE value, the higher the prediction accuracy of the triple exponential smoothing.

As can be seen from Figure 7, for triple exponential smoothing, the optimal smoothing factors of Mysql CPU load and Nginx CPU load are both 0.1, while the optimal smoothing factor of Redis CPU load is 0.2. The optimal smoothing factors of Mysql memory load and Nginx memory load are both 0.2, while the optimal smoothing factor of Redis memory load is 0.4. This is because their load is relatively stable, and there is no long-term change trend. Since Redis is used as a cache service that supports higher-speed data read and write, its load has a clear short-term trend. So the optimal smoothing factor of Redis is larger than the other two application containers. We only compare the smoothing factors of different applications of the same load type.

The hybrid model is usually the best when the triple exponential smoothing is not optimal. This is because the smoothing factor does not affect LSTM, and we combine two models using the mean absolute percentage error(MAPE) method. Triple exponential smoothing has advantages when the amount of data is small. With more and more data, the advantages of LSTM become more evident. Because we consider the average MAPE of all previous predicted data, the optimal smoothing factor of the hybrid model is different from the optimal smoothing factor of the triple exponential smoothing.

The parameters of LSTM: for LSTM, we need to specify its feature, timestamp, and learning rate η . The dimension of the input data determines the feature of LSTM. Since the input of the prediction model is only the previous load which is one-dimensional data, we set the feature to 1. Timestamp determines the number of historical values to predict the next value. Through many experiments, we find that the timestamp does not affect the accuracy of LSTM, so we set the timestamp to 1. For learning rate η , we set it to the default value of 0.01 as in Keras [39]. After conducting many experiments, the optimal training epoch we choose is 100.

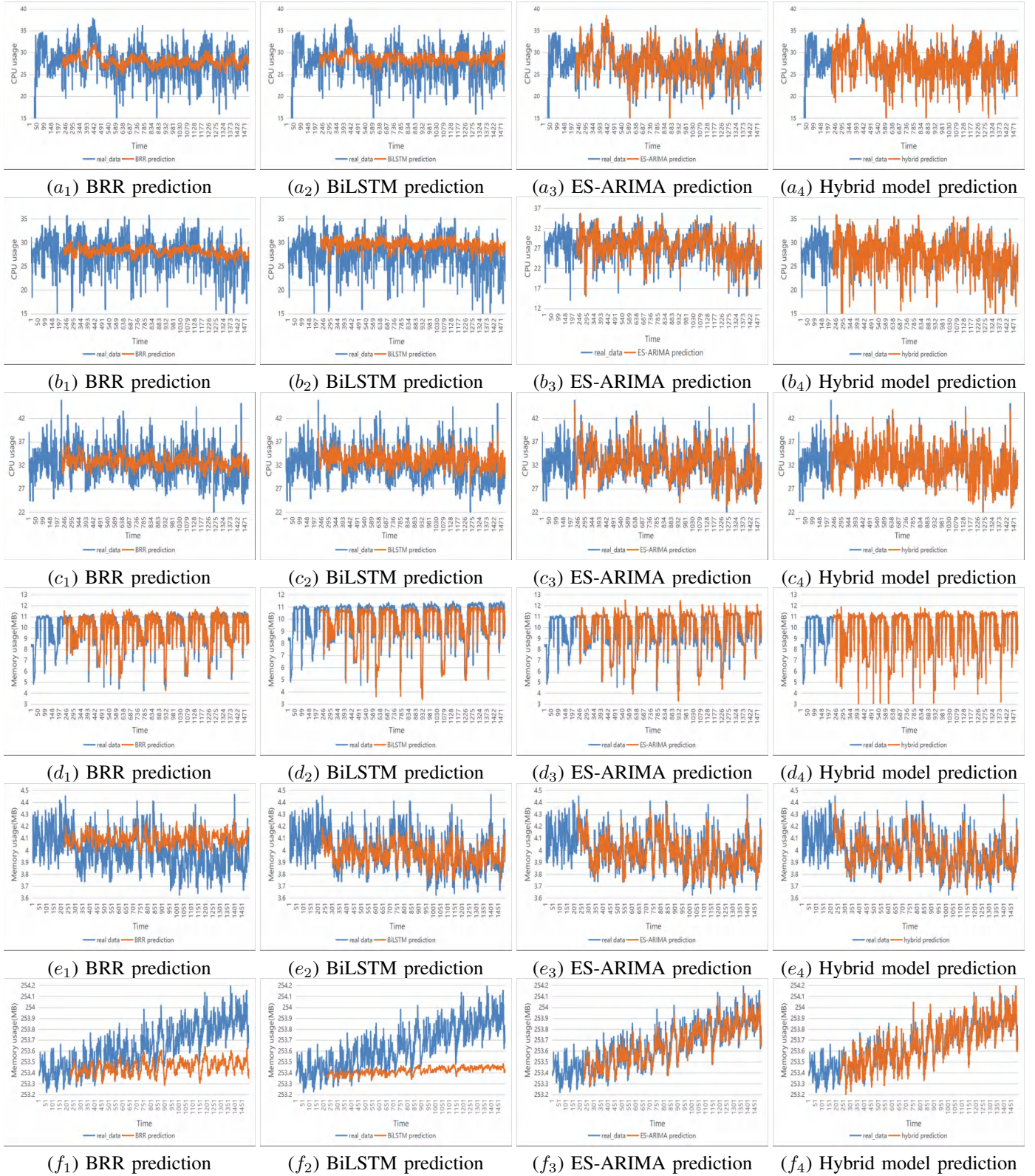


Fig. 8: The prediction effectiveness chart of four models under diverse application container workloads in the simulated environment. ($a_1 - a_4$) shows the prediction effectiveness of Redis CPU load, ($b_1 - b_4$) shows the prediction effectiveness of Nginx CPU load, ($c_1 - c_4$) shows the prediction effectiveness of Mysql CPU load. ($d_1 - d_4$) shows the prediction effectiveness of Redis memory load, ($e_1 - e_4$) shows the prediction effectiveness of Nginx memory load, ($f_1 - f_4$) shows the prediction effectiveness of Mysql memory load. The blue line displays the actual CPU utilization or memory utilization and the orange line displays the load prediction result of the diverse prediction models. BRR stands for Bayesian Ridge Regression.

TABLE V: Prediction result in the simulated environment

Load type	Prediction model	MAPE	MAE
Redis CPU load	LSTM	8.863%	2.326
	triple exponential smoothing	9.441%	2.498
	hybrid model	8.006%	2.117
	ES-ARIMA	9.486%	2.506
	Bayesian Ridge Regression	8.913%	2.496
	BiLSTM	8.867%	2.529
Nginx CPU load	LSTM	9.015%	2.324
	triple exponential smoothing	9.809%	2.552
	hybrid model	8.292%	2.157
	ES-ARIMA	9.868%	2.574
	Bayesian Ridge Regression	8.700%	2.444
	BiLSTM	9.496%	2.789
Mysql CPU load	LSTM	7.318%	2.366
	triple exponential smoothing	8.099%	2.617
	hybrid model	6.923%	2.230
	ES-ARIMA	8.081%	2.588
	Bayesian Ridge Regression	8.185%	2.682
	BiLSTM	7.425%	2.457
Redis memory load	LSTM	5.565%	0.462
	triple exponential smoothing	7.322%	0.615
	hybrid model	5.140%	0.411
	ES-ARIMA	7.811%	0.678
	Bayesian Ridge Regression	5.403%	0.504
	BiLSTM	5.419%	0.512
Nginx memory load	LSTM	2.032%	0.081
	triple exponential smoothing	2.111%	0.084
	hybrid model	1.885%	0.075
	ES-ARIMA	1.939%	0.077
	Bayesian Ridge Regression	3.577%	0.146
	BiLSTM	2.026%	0.080
Mysql memory load	LSTM	0.030%	0.00078
	triple exponential smoothing	0.024%	0.00062
	hybrid model	0.021%	0.00054
	ES-ARIMA	0.023%	0.00100
	Bayesian Ridge Regression	0.096%	0.00240
	BiLSTM	0.084%	0.00210

C. Prediction accuracy comparison

To compare the prediction accuracy, six diverse prediction methods are used. They are triple exponential smoothing model, LSTM model, the hybrid model, a workload prediction model using triple exponential smoothing and autoregressive integrated moving average algorithm [23] which we call ES-ARIMA model, Bayesian Ridge Regression [21] and a Bidirectional LSTM model [40] which we call BiLSTM model.

Since CPU and memory are the two most commonly used resources in elastic scaling and pay-as-you-go service, we use CPU and memory load to verify the prediction accuracy. We use mean absolute percentage error (MAPE) and mean absolute

error (MAE) to assess the method prediction performance. The smaller the MAPE and MAE, the higher the prediction accuracy of the method. Their formulas are shown below:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{X_i - Y_i}{X_i} \right| \quad (23)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |X_i - Y_i| \quad (24)$$

Where X_i refers to the real value and Y_i refers to the prediction value.

TABLE VI: Prediction result of the Alibaba cluster-trace-v2018. “MAPE reduction rate” represents the MAPE reduction rate of the hybrid model compared to other models. “MAE reduction rate” represents the MAE reduction rate of the hybrid model compared to other models.

Prediction model	MAPE	MAE	MAPE reduction rate	MAE reduction rate
triple exponential smoothing	20.647%	1.544	11.08%	9.326%
LSTM	18.587%	1.484	1.226%	5.660%
ES-ARIMA	20.830%	1.544	11.862%	9.326%
Bayesian Ridge Regression	35.579%	2.235	48.399%	37.36%
BiLSTM	41.908%	2.324	56.192%	39.759%
hybrid model	18.359%	1.400	0%	0%

1) *Experiment in the simulated environment*: We compare the prediction results of the six models under diverse application container loads in the simulated environment. The prediction results are shown in Table V. LSTM and triple exponential smoothing have lower prediction accuracy than the hybrid model. This is because the hybrid model uses the mean absolute percentage error (MAPE) method to combine those two models and plays to the strengths of both models at the same time. In comparison to the ES-ARIMA model, the prediction accuracy of the hybrid model is higher because LSTM can better capture the workload characteristics than ARIMA. ARIMA is a model that mines the linear relationship between load data items, while LSTM can exploit short-term and long-term temporal dependencies in the time series. The hybrid model also performs better than Bayesian Ridge Regression and BiLSTM because the hybrid model is an online update model and can timely capture load change trends.

Figure 8 further shows the original data and the predicted values of the four models under diverse application container loads in the simulated cloud environment. Each load data contains 1500 seconds of load. In Figure 8, the prediction begins at the 226th point. This is because we set fifteen percent of load data as the model's pre-training data, and the remaining 1,275 seconds of load is used as the prediction target. Among the three application workloads, Mysql has the highest average CPU and memory usage, while Redis has the lowest average CPU usage. This is because a small amount of data is read and written from the cache, and most of the data is read and written from the database. Nginx has the lowest average memory usage. This is because Nginx processes HTTP requests and static resource requests, occupying less memory. Since Redis is a cache service that supports higher-speed data read and write, the CPU and memory usage have the largest range and short-term trend.

In Table V, the CPU load prediction performance of ES-ARIMA is worse than BiLSTM and Bayesian Ridge Regression, which is a little different from the results in the Figure 8. This is because the CPU load changes sharply, resulting in a lag in ES-ARIMA prediction. As the prediction effectiveness chart contains load data for 1,500 time points, the prediction lag is not visible in Figure 8.

2) *Experiment using the Alibaba cluster-trace-v2018*: We conduct the experiment using the six models to measure all container usage in Alibaba-cluster-data-v2018 and calculate the average prediction result of the diverse six models. The

prediction result is shown in Table VI. Because the time interval of cluster-data-v2018 resampled is ten minutes, we set the smoothing factor of the hybrid model to 0.3 by comparing the experiment results. The experiment results show that the prediction accuracy of the hybrid model is the best, which is consistent with the experimental results in the simulated environment. This is not only because the simulated environment we build is closer to the real cloud environment but also because the hybrid model can capture the characteristics of more types of container loads.

By analyzing the container usage of Alibaba cluster-data-v2018, we find that its container load has the following characteristics: 1) **periodic**, 2) **load change rapidly**, 3) **stability**. To explain the performance of different models under different characteristics, we select six different typical container loads to interpret experimental results, which are shown in Table VII and Figure 9.

periodic: For container c_10104, because it has enough data and the training data includes a complete cycle of load, several models perform well. The hybrid model performs the best. This is because LSTM has advantages in periodic time series prediction and can capture long-term and short-term dependencies, and triple exponential smooth can capture data trends. The hybrid model combines the best of both models. For c_10044 and c_10042, though the amount of training data is small and training data does not contain a complete cycle of load, the hybrid model performs the best because it can be dynamically updated based on real-time load data. For c_1020, the load data of this container is periodic from about the 183rd point. The real-time updated hybrid model performs the best while Bayesian Ridge Regression and BiLSTM perform poorly. This is because Bayesian Ridge Regression and BiLSTM cannot capture the periodicity of subsequent load data in time.

load changes rapidly: For c_10063, the load of this container changes rapidly. When the load suddenly rises from very low to a relatively high level, the predicted value of triple exponential smoothing will drop for a while, and then rise. This indicates that triple exponential smoothing has a poor ability to capture rapid changes. Because of the poor prediction accuracy of triple exponential smoothing, the prediction accuracy of the hybrid model is slightly lower than that of the LSTM. However, both the hybrid model and the LSTM model perform better compared with Bayesian Ridge Regression and BiLSTM because the LSTM model can capture the long-term

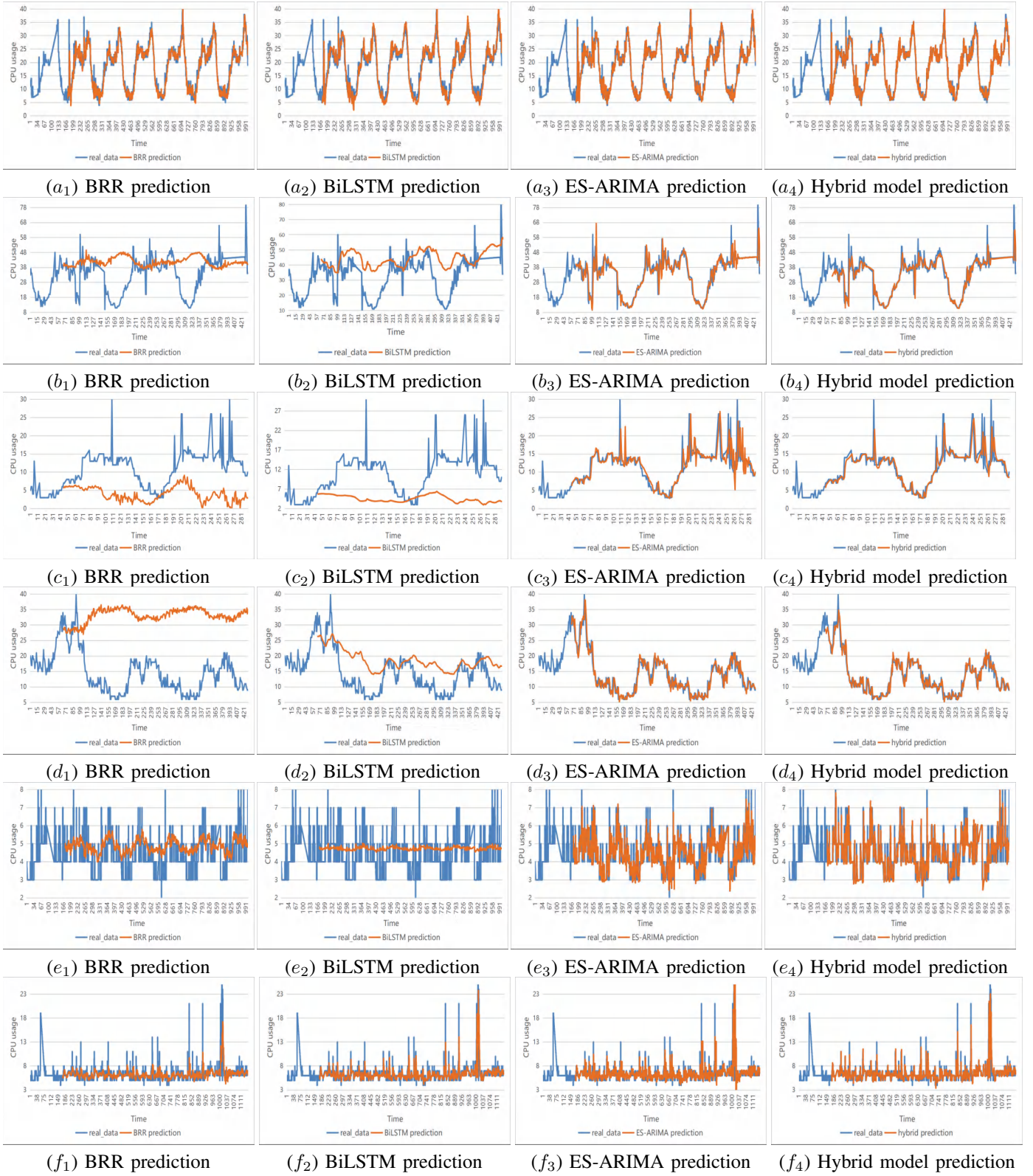


Fig. 9: The prediction effectiveness chart of four models under six diverse containers in Alibaba cluster-trace-v2018. ($a_1 - a_4$) shows the prediction effectiveness of container c_{10104} , ($b_1 - b_4$) shows the prediction effectiveness of container c_{10044} , ($c_1 - c_4$) shows the prediction effectiveness of container c_{10042} , ($d_1 - d_4$) shows the prediction effectiveness of container c_{1020} , ($e_1 - e_4$) shows the prediction effectiveness of container c_{10063} , ($f_1 - f_4$) shows the prediction effectiveness of container c_{10001} . The blue line displays the actual CPU utilization and the orange line displays the prediction result of the diverse models. BRR stands for Bayesian Ridge Regression.

TABLE VII: Prediction result using Alibaba cluster data-v2018 typical container workload. “Length” represents how many time points of load data a container has.

Container ID	Characteristics	Prediction model	MAPE	MAE
c_10104	large amount of data (length = 999) and periodic	LSTM	12.114%	1.955
		triple exponential smoothing	12.226%	1.964
		hybrid model	11.345%	1.829
		ES-ARIMA	11.887%	1.933
		Bayesian Ridge Regression	13.998%	1.971
		BiLSTM	16.447%	2.165
c_10044	little amount of data (length = 433) and periodic	LSTM	11.948%	3.907
		triple exponential smoothing	10.268%	3.348
		hybrid model	9.385%	3.037
		ES-ARIMA	10.256%	3.327
		Bayesian Ridge Regression	25.123%	10.936
		BiLSTM	26.124%	11.126
c_10042	periodicity with rapid change (length = 289)	LSTM	12.528%	1.599
		triple exponential smoothing	14.137%	1.756
		hybrid model	11.859%	1.490
		ES-ARIMA	14.445%	1.816
		Bayesian Ridge Regression	685.375%	7.937
		BiLSTM	176.122%	7.458
c_1020	partial periodicity	LSTM	10.543%	1.510
		triple exponential smoothing	11.312%	1.487
		hybrid model	10.154%	1.342
		ES-ARIMA	11.024%	1.457
		Bayesian Ridge Regression	57.988%	19.623
		BiLSTM	30.905%	5.572
c_10063	load change rapidly	LSTM	14.818%	0.713
		triple exponential smoothing	17.172%	0.809
		hybrid model	14.936%	0.715
		ES-ARIMA	15.505%	0.750
		Bayesian Ridge Regression	19.714%	0.940
		BiLSTM	19.034%	0.903
c_10001	stability	LSTM	14.218%	0.989
		triple exponential smoothing	15.782%	1.073
		hybrid model	13.853%	0.958
		ES-ARIMA	14.504%	1.009
		Bayesian Ridge Regression	14.165%	0.950
		BiLSTM	13.785%	0.950

and short-term dependencies in such rapid changes of load.

stability: For c_10001, the change range of this load is small, and there is no obvious long-term and short-term trend. The prediction accuracy of the hybrid model is better than other models, second only to BiLSTM. To explain the reason, we intercept a fragment of the c_10001 prediction effectiveness chart as shown in the Figure 10. The hybrid model is relatively accurate in predicting the changing trend of container load, but its prediction has a lag. When the hybrid model is about to predict its changing trend, the load changes again, resulting in the fact that the prediction accuracy of the

hybrid model is not the highest. Since BiLSTM has a relatively poor predictive ability, it is not greatly affected by the lag and has a smaller error than the hybrid model. Compared with other models except BiLSTM, the hybrid model can be updated in real-time and thus can adjust from this error faster, so its accuracy is second only to BiLSTM.

To sum up, by comparing the prediction results of different methods on Alibaba cluster-data-v2018 container usage data, the hybrid model performs better because it can not only adjust according to real-time data in time but also can capture long-term and short-term dependencies and capture data trends at

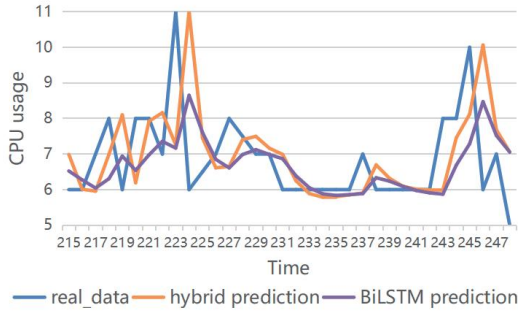


Fig. 10: Partial prediction effectiveness chart of the container c_10001. Although the jitter of container resource usage is small, we enlarge the chart to see it more clearly.

the same time. Specifically, Table VI shows that the mean absolute percentage error of the hybrid model is decreased by an average of 1.226%, 11.08%, 11.862%, 48.399%, and 56.192% compared with the LSTM, the triple exponential smoothing, ES-ARIMA, BiLSTM and Bayesian Ridge Regression on Alibaba cluster-data-v2018 container usage data, which indicates the prediction accuracy of the hybrid model has improved a lot compared with other methods.

3) *Comparative analysis of experimental results:* The experimental results show that the prediction performance of the hybrid model is good in both the simulated cloud environment and the Alibaba cluster-trace-v2018. In the simulated environment for different application loads, the prediction effect of the hybrid model is the best. Under Alibaba cluster-trace-v2018, the hybrid model performs the best overall.

Compared with the simulated environment, the Alibaba cluster-trace-v2018 contains more load features, and the amount of load data varies. It can be seen that there are more or less repetitive patterns in the load in Figure 9. When there are few repetitive patterns or less amount of load data, the hybrid model can also dynamically update to adapt to the load changes, for which the hybrid model can perform very well.

Unlike the simulated environment, there are a few cases in the Alibaba cluster-trace-v2018 where the load changes and fluctuations are more severe than in the simulated cloud environment. The real cloud environment may have some abnormal load due to the attack. These situations may lead to a relatively strong lag in the hybrid model. The hybrid model can adapt to these situations with the real-time update.

D. Prediction time

For a proactive prediction method, its prediction time must be short to guarantee that predictions are available before resource allocation. We record the prediction time of LSTM, hybrid model, ES-ARIMA, and BiLSTM and show the average time it takes to predict a value in Figure 11. Figure 11 shows the prediction time of LSTM accounts for the most prediction time of the hybrid model. This is because the prediction value of LSTM needs to be obtained by using more complex matrix operations while the calculation of triple exponential smoothing is relatively simple. So the hybrid model has a comparable prediction time with LSTM.

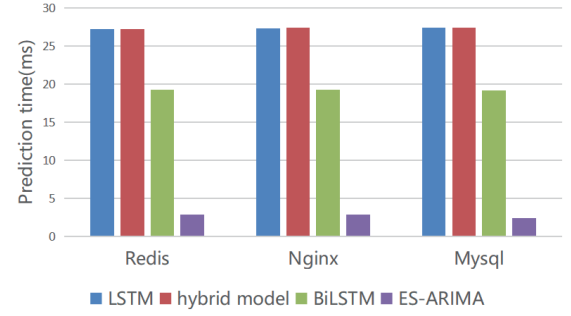


Fig. 11: Average prediction time contrast among the LSTM, the hybrid model, BiLSTM, and ES-ARIMA.

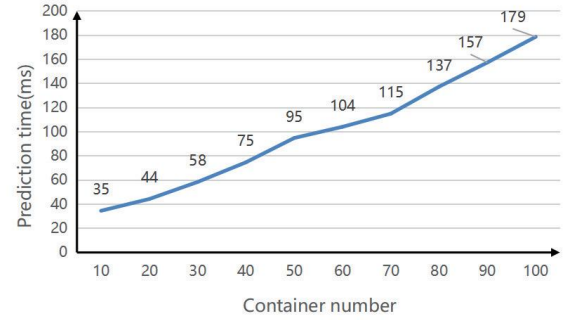


Fig. 12: Average prediction time for diverse numbers of containers.

The prediction time of LSTM is longer than BiLSTM. This is because our LSTM model is an online update model which needs time to update before prediction. Since the prediction time of LSTM is longer than BiLSTM, the prediction time of the hybrid model is longer than BiLSTM. The prediction time of ES-ARIMA is shorter than the hybrid model. This is because ES-ARIMA is a computational model while the hybrid model contains a deep learning model. However, the hybrid model consumes time only in milliseconds but with high prediction accuracy. In the prediction system, the collection, prediction, and scheduling intervals are the same, all in seconds. From the experimental result, the hybrid model meets the requirement of the prediction with interval in seconds.

E. Scalability analysis

When the number of containers increases, the load data of more containers needs to be maintained for resource prediction and resource allocation. This requires more processes to handle. At the same time, the contention and preemption of resources between different containers will be more intense, which may affect the system performance.

We conduct experiments under different numbers of containers and find the number of containers has no impact on the prediction accuracy. We record the average prediction time for different numbers of containers as shown in Figure 12. When the number of containers increases, the prediction time also increases linearly and the prediction time is acceptable. This is because we adopt a multi-threading strategy to run multiple containers and the model we used is lightweight. The experimental results show that the hybrid model has very good scalability.

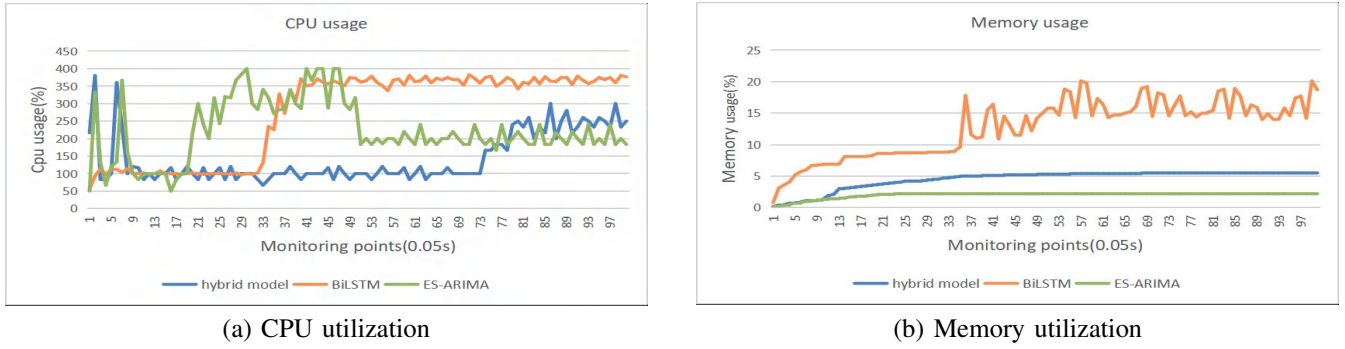


Fig. 13: The CPU and memory consumption of diverse models.

TABLE VIII: Summary of related research

Ref	Objective	Methods	Advantages	Limitations
[23]	Container	TES-ARIMA	Real-time and high robustness	Accuracy not high enough
[24]	Container	SG+LSTM+PSO	automatic hyper-parameter tuning	Low accuracy and high computational complexity
[41]	Containerized edge computing	PRP+PRCT	Low service latency	High time complexity
[19]	Cloud resource	Naive Bayesian	High prediction accuracy	High computational costs
[42]	Cloud resource	Statistical learning+Sliding window	Dynamic Adjustment	High computational costs
[20]	Cloud resource	ARIMA	High prediction accuracy	Limited workload scenarios
[13]	Cloud resource	Bayesian	Good prediction accuracy	High complexity
[43]	Cloud resource	ARIMA+Autoregressive Neural Network	Adaptability for resource	Accuracy not high enough
[44]	Cloud resource	SaDE+ANN	Reducing iterations	High computational costs
[21]	Cloud resource	PBC & DBC	High prediction accuracy	High computational costs
[22]	Cloud resource	SVM+BP	Hybrid model	Low prediction accuracy
[45]	Cloud resource	pCNN-LSTM	High prediction accuracy	High computational costs

F. CPU and memory overhead

Figure 13 shows the CPU and Memory usage of running ES-ARIMA, BiLSTM, and the hybrid model using the Alibaba cluster-trace-v2018. The CPU and Memory utilization are recorded every 0.05 seconds. The experimental results show CPU utilization of the hybrid model is lower than the ES-ARIMA most of the time. This is because the LSTM network which we use is very lightweight and the parameters of ARIMA need to be determined through multiple attempts during the running process of the program. The Memory usage of the hybrid model is slightly higher than ES-ARIMA. This is because the hybrid model contains a deep learning model while ES-ARIMA is equivalent to some calculation formulas. Due to the bidirectional structure of BiLSTM, its resource consumption is higher than that of the hybrid model. All in all, the hybrid model can achieve higher accuracy with a similar computational cost to ES-ARIMA.

VI. RELATED WORK

In this section, we introduce the related research on the load prediction method containing the Docker load prediction method and cloud resource prediction method. We also summarize these research in Table VIII.

A. Docker load prediction method

There are only a few related studies on Docker container workload prediction [23], [24], [41]. The characteristics of the Docker container, such as short running period and elastic scaling, make container load prediction need to take into account the short-term prediction and the load with severe fluctuations. In a container cluster, the number of containers is large, which requires the prediction algorithm to be scalable. At the same time, container load prediction faces the problems of real-time scheduling and dynamic load changes. In order to feed back the prediction results to the system in time, the prediction should be in real-time. Moreover, since the changes in container load cannot be fully predicted in advance, the prediction method needs to adjust based on the load changes in time.

Xie *et al.* [23] proposed a hybrid model of the ARIMA and triple exponential smoothing, which can capture both linear and nonlinear relationships in the container workload data. However, this hybrid model is a simple combination of the ARIMA and triple exponential smoothing, whose prediction accuracy is not good enough. Chen *et al.* [24] presented a container workload prediction approach called PSL. Hu *et al.* [41] introduced a dynamic resource provisioning approach

CEC, which contains a workload prediction method and resource allocation method in an edge computing environment. These two methods do not consider the characteristics of the container load. Moreover, applying these methods to a real-time prediction scenario is a difficult problem.

B. Cloud resource prediction

There are a few studies on cloud resource prediction, mainly through traditional time series prediction methods, to predict load based on time series and cloud resource characteristics. Calheiros *et al.* [20] presented a cloud workload prediction module based on the ARIMA model for SaaS providers. Ullah *et al.* [43] proposed an adaptive system for resource utilization prediction of IaaS cloud, which takes real-time resource utilization and feeds resource utilization data into several buffers based on the resource type and period size. ARIMA is an excellent time series prediction method that can capture the linear relationship in time series. However, the features that ARIMA can capture are limited. Moreover, ARIMA requires that the time series must be stationary. Otherwise, the time series must do difference operation.

Machine learning and deep learning methods have good feature capture capabilities, so many researchers apply those methods to load prediction. Bey *et al.* [19] proposed a method based on Fuzzy clustering and multiple system workload predictors controlled through a Naive Bayesian Network. Islam *et al.* [42] proposed a prediction framework using statistical learning algorithms and a sliding window mechanism. Shyam *et al.* [13] proposed a Bayesian model to determine short and long-term virtual cloud resource requirements of the CPU/memory intensive application. KUMAR *et al.* [44] introduced a cloud workload prediction method that uses the self-adaptive differential evolution algorithm(SaDE) to train the neural network. Gao *et al.* [21] introduced a workload prediction idea that clusters all the data into several categories by clustering and training a workload prediction model for each workload type, respectively. Sun *et al.* [22] introduced a cloud workload prediction method combining SVM and BP neural networks. Patel *et al.* [45] presented a novel hybrid workload prediction model called pCNN-LSTM. These methods have strong feature capture ability and relatively high prediction accuracy. However, these methods do not solve the problem of real-time prediction and usually have a high overhead.

VII. DISCUSSION AND LIMITATIONS

We do not test the performance of the hybrid model on virtual machine load data because we focus on research on container load prediction. The applicability of our method in virtual machine load prediction is as follows. First, both virtual machine load prediction and container load prediction are time series prediction, and virtual machine load has periodic characteristics like container load [16], [17]. Second, the amount of training data for virtual machine load prediction is relatively large. This can ensure that our method has better performance. Third, our method can adapt to the container environment with more fluctuations in load. We can also adapt our method to the virtual machine load with less fluctuation

by adjusting the *timestamp* and *smoothing factor*. Considering the above factors, we believe that our method will also have a good effect on virtual machine load prediction.

We do not optimize the parameters of LSTM. According to our experimental results, the current parameters are suitable. Regarding LSTM parameter optimization ideas, we can use both a dynamic learning rate strategy and grid search [46] to optimize the parameters of LSTM automatically.

The limitations of our method are as below:

- 1) The hybrid model prediction has a lag when the workload change range is small and rapid.
- 2) We mainly predict CPU and memory utilization in this work. We will try to predict and allocate other resources in future work.
- 3) Our allocation strategy uses the range of load resources ($\text{Max}(x_t) - \text{Min}(x_t)$) as the worst prediction error, which is a little conservative. We will improve and justify our allocation strategy in future work.

VIII. CONCLUSION

With the increasing popularization of container technology on cloud platforms, efficient resource management becomes challenging for cloud service providers. We propose a container workload proactive prediction method using a hybrid model combining triple exponential smoothing and LSTM. The hybrid model can capture both short-term and long-term dependencies and smooth the container load data. We also optimize the hybrid model update to overcome the load pattern change. For the prediction of resource utilization, the mean absolute percentage error has decreased by an average of 3.24%, 12.18%, 13.42%, 43.45%, and 50.69% compared with the LSTM, the triple exponential smoothing model, ES-ARIMA algorithm, Bayesian Ridge Regression, and BiLSTM. What is more, the time and computational cost overhead of the hybrid model is acceptable.

Our method helps to achieve elastic scaling, improve resource utilization, reduce resource waste, and ensure pay-as-you-go service, which can effectively promote the development of container cloud platforms. Cloud service providers can use the container workload prediction system to perform container workload prediction on the Docker-based platform to provide efficient resource prediction and provisioning. This can greatly improve cloud platform service quality and improve the user experience of using the container cloud platform.

ACKNOWLEDGMENT

This work was supported in part by the National Key R&D Program of China under Grant No.2022YFB4501300 and the National Science Foundation of China under Grant No.61972449, 61972448, and 61821003. Corresponding author: Yulai Xie, ylxie@hust.edu.cn

REFERENCES

- [1] C. Anderson, "Docker [software engineering]," *Ieee Software*, vol. 32, no. 3, pp. 102–c3, 2015.

- [2] J. N. Acharya and A. C. Suthar, "Docker container orchestration management: A review," in *Proceedings of the International Conference on Intelligent Vision and Computing (ICIVC 2021)*, H. Sharma, V. K. Vyas, R. K. Pandey, and M. Prasad, Eds. Cham: Springer International Publishing, 2022, pp. 140–153.
- [3] E. Casalicchio, "Container orchestration: a survey," *Systems Modeling: Methodologies and Tools*, pp. 221–235, 2019.
- [4] K. Ye and Y. Ji, "Performance tuning and modeling for big data applications in docker containers," in *2017 International Conference on Networking, Architecture, and Storage (NAS)*. Los Alamitos, CA, USA: IEEE Computer Society, aug 2017, pp. 1–6.
- [5] V. Medel, O. Rana, J. a. Bañares, and U. Arronategui, "Modelling performance amp; resource management in kubernetes," in *Proceedings of the 9th International Conference on Utility and Cloud Computing*, ser. UCC '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 257–262.
- [6] K. Ye, Y. Kou, C. Lu, Y. Wang, and C.-Z. Xu, "Modeling application performance in docker containers using machine learning techniques," in *2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS)*, 2018, pp. 1–6.
- [7] S. M. Raza, J. Jeong, M. Kim, B. Kang, and H. Choo, "Empirical performance and energy consumption evaluation of container solutions on resource constrained iot gateways," *Sensors*, vol. 21, no. 4, 2021.
- [8] G. Iswari, R. W. Sudibyo, H. A. Darwito, and M. M. Islam, "Network performance evaluation of container server-based lorawan iot for field worker monitoring system," in *2021 International Electronics Symposium (IES)*, 2021, pp. 653–657.
- [9] X. Guan, X. Wan, B.-Y. Choi, S. Song, and J. Zhu, "Application oriented dynamic resource allocation for data centers using docker containers," *IEEE Communications Letters*, vol. 21, no. 3, pp. 504–507, 2017.
- [10] Y. Li and Y. Xia, "Auto-scaling web applications in hybrid cloud based on docker," in *2016 5th International Conference on Computer Science and Network Technology (ICCSNT)*, 2016, pp. 75–79.
- [11] A. A. Khudur and R. P. Polos, "Gathering multi-dimensional data of scalable container-based cluster for performance analysis," in *2021 Fourth International Conference on Computational Intelligence and Communication Technologies (CCICT)*, 2021, pp. 139–145.
- [12] J. Kumar, A. K. Singh, and R. Buyya, "Self directed learning based workload forecasting model for cloud resource management," *Information Sciences*, vol. 543, pp. 345–366, 2021.
- [13] G. K. Shyam and S. S. Manvi, "Virtual resource prediction in cloud environment: a bayesian approach," *Journal of Network and Computer Applications*, vol. 65, pp. 144–154, 2016.
- [14] A. Inc, "Alibaba production cluster data v2018," 2018. [Online]. Available: <https://github.com/alibaba/clusterdata/tree/v2018>
- [15] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long-and short-term temporal patterns with deep neural networks," in *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, 2018, pp. 95–104.
- [16] A. Y. Nikraves, S. A. Ajila, and C.-H. Lung, "An autonomic prediction suite for cloud resource provisioning," *J. Cloud Comput.*, vol. 6, no. 1, dec 2017.
- [17] M. E. Karim, M. M. S. Maswood, S. Das, and A. G. Alharbi, "Bhyprec: A novel bi-lstm based hybrid recurrent neural network model to predict the cpu workload of cloud virtual machine," *IEEE Access*, vol. 9, pp. 131 476–131 495, 2021.
- [18] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux journal*, vol. 2014, no. 239, p. 2, 2014.
- [19] F. Benhamadi, Z. Gessoum, A. Mokhtari *et al.*, "Cpu load prediction using neuro-fuzzy and bayesian inferences," *Neurocomputing*, vol. 74, no. 10, pp. 1606–1616, 2011.
- [20] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using arima model and its impact on cloud applications' qos," *IEEE Transactions on Cloud Computing*, vol. 3, no. 4, pp. 449–458, 2014.
- [21] J. Gao, H. Wang, and H. Shen, "Machine learning based workload prediction in cloud computing," in *2020 29th International Conference on Computer Communications and Networks (ICCCN)*, 2020.
- [22] Q. Sun, Z. Tan, and X. Zhou, "Workload prediction of cloud computing based on svm and bp neural networks," *Journal of Intelligent Fuzzy Systems*, vol. 39, pp. 2861–2867, 10 2020.
- [23] Y. Xie, M. Jin, Z. Zou, G. Xu, D. Feng, W. Liu, and D. Long, "Real-time prediction of docker container resource load based on a hybrid model of arima and triple exponential smoothing," *IEEE Transactions on Cloud Computing*, 2020.
- [24] L. Chen and W. Zhang, "A deep learning-based approach with pso for workload prediction of containers in the cloud," in *2021 13th International Conference on Advanced Infocomm Technology (ICAIT)*, 2021, pp. 204–208.
- [25] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," 1999.
- [26] C. Chatfield, "The holt-winters forecasting procedure," *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 27, no. 3, pp. 264–279, 1978.
- [27] A. De Myttenaere, B. Golden, B. Le Grand, and F. Rossi, "Mean absolute percentage error for regression models," *Neurocomputing*, vol. 192, pp. 38–48, 2016.
- [28] E. S. Gardner Jr, "Exponential smoothing: The state of the art," *Journal of forecasting*, vol. 4, no. 1, pp. 1–28, 1985.
- [29] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [30] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010*. Springer, 2010, pp. 177–186.
- [31] "Glances," <https://nicolargo.github.io/glances/>.
- [32] S. N. Z. Naqvi, S. Yfantidou, and E. Zimányi, "Time series databases and influxdb," *Studienarbeit, Université Libre de Bruxelles*, 2017.
- [33] "Redis," <https://redis.io/>.
- [34] "Nginx," <https://www.nginx.com/>.
- [35] "Mysql," <https://www.mysql.com/>.
- [36] "Djangoblog," <https://github.com/liangliangyy/DjangoBlog/>.
- [37] "http_load," https://acme.com/software/http_load/.
- [38] J. Guo, Z. Chang, S. Wang, H. Ding, Y. Feng, L. Mao, and Y. Bao, "Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces," in *Proceedings of the International Symposium on Quality of Service*. Association for Computing Machinery, 2019.
- [39] "Keras," <https://keras.io/>.
- [40] D. Ruta, L. Cen, and Q. H. Vu, "Deep bi-directional lstm networks for device workload forecasting," in *2020 15th Conference on Computer Science and Information Systems (FedCSIS)*, 2020.
- [41] S. Hu, W. Shi, and G. Li, "Cec: A containerized edge computing framework for dynamic resource provisioning," *IEEE Transactions on Mobile Computing*, pp. 1–1, 2022.
- [42] S. Islam, J. Keung, K. Lee, and A. Liu, "Empirical prediction models for adaptive resource provisioning in the cloud," *Future Generation Computer Systems*, vol. 28, no. 1, pp. 155–162, 2012.
- [43] Q. Zia Ullah, S. Hassan, and G. M. Khan, "Adaptive resource utilization prediction system for infrastructure as a service cloud," *Computational intelligence and neuroscience*, vol. 2017, 2017.
- [44] J. Kumar and A. K. Singh, "Workload prediction in cloud using artificial neural network and adaptive differential evolution," *Future Generation Computer Systems*, vol. 81, pp. 41–52, 2018.
- [45] E. Patel and D. S. Kushwaha, "A hybrid cnn-lstm model for predicting server load in cloud computing," *The Journal of Supercomputing*, pp. 1–30, 2022.
- [46] P. Lerman, "Fitting segmented regression models by grid search," *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 29, no. 1, pp. 77–84, 1980.



Liangkang Zhang received the B.E. degree in information security from Lanzhou University, China, in 2021. He is currently pursuing his master's degree in Huazhong University of Science and Technology (HUST). His research interests include Docker container and virtualization.



Yulai Xie received the B.E. and Ph.D. degrees in computer science from Huazhong University of Science and Technology (HUST), China, in 2007 and 2013, respectively. He was a visiting scholar at the University of California, Santa Cruz in 2010 and a visiting scholar at the Chinese University of Hong Kong in 2015. He is now an professor in School of Cyber Science and Engineering in HUST, China. His research interests mainly include cloud storage and virtualization, digital provenance, intrusion detection, machine learning and computer

architecture.



Yafeng Wu received master's degree in computer science from Huazhong University of Science and Technology in 2019. He is currently pursuing his Ph.D. degree in Huazhong University of Science and Technology (HUST). His research interests mainly include digital provenance and intrusion detection.



Minpeng Jin received the B.E. degree in computer science from Northeastern University, China, in 2019 and master's degree in Huazhong University of Science and Technology (HUST) in 2022, respectively.



Dan Feng received her B.E, M.E. and Ph.D. degrees in Computer Science and Technology from Huazhong University of Science and Technology (HUST), China, in 1991, 1994 and 1997 respectively. She is a professor and director of Data Storage System Division, Wuhan National Lab for Optoelectronics. She is also dean of the School of Computer Science and Technology, HUST. Her research interests include computer architecture, massive storage systems, parallel file systems, disk array and solid state disk. She has over 100 publications in journals and international conferences, including FAST, USENIX ATC, ICDCS, HPDC, SC, ICS and IPDPS. Dr. Feng is a senior member of IEEE and a member of ACM.

nals and international conferences, including FAST, USENIX ATC, ICDCS, HPDC, SC, ICS and IPDPS. Dr. Feng is a senior member of IEEE and a member of ACM.



Pan Zhou is currently a professor with Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology (HUST), Wuhan, P.R. China. He received his Ph.D. in the School of Electrical and Computer Engineering at the Georgia Institute of Technology (Georgia Tech) in 2011, Atlanta, USA. He received his B.S. degree in the Advanced Class of HUST, and a M.S. degree in the Department of Electronics and Information Engineering from HUST, Wuhan, China, in

2006 and 2008, respectively. He held honorary degree in his bachelor and merit research award of HUST in his master study. He was a senior technical member at Oracle Inc., America, during 2011 to 2013, and worked on Hadoop and distributed storage system for big data analytics at Oracle Cloud Platform. He received the "Rising Star in Science and Technology of HUST" in 2017. His current research interest includes: security and privacy, big data analytics and machine learning, and information networks.



Darrell Long received his B.S. degree in Computer Science from San Diego State University, and his M.S. and Ph.D. from the University of California, San Diego. Dr. Darrell D.E. Long is Distinguished Professor of Computer Engineering at the University of California, Santa Cruz. He holds the Kumar Malavalli Endowed Chair of Storage Systems Research and is Director of the Storage Systems Research Center. His current research interests in the storage systems area include high performance storage systems, archival storage systems and energy-efficient storage systems. His research also includes computer system reliability, video-on-demand, applied machine learning, mobile computing and cyber security. Dr. Long is Fellow of IEEE and Fellow of the American Association for the Advancement of Science (AAAS).

efficient storage systems. His research also includes computer system reliability, video-on-demand, applied machine learning, mobile computing and cyber security. Dr. Long is Fellow of IEEE and Fellow of the American Association for the Advancement of Science (AAAS).



Gongming Xu received the B.E. degree in computer science from Wuhan Institute of Technology, China, in 2018 and master's degree in Huazhong University of Science and Technology (HUST) in 2020, respectively.